



UMĚLÁ INTELIGENCE

Přednáška 5

Soupeřivé prohledávání a hry

SOUPEŘIVÉ PROHLEDÁVÁNÍ A HRY

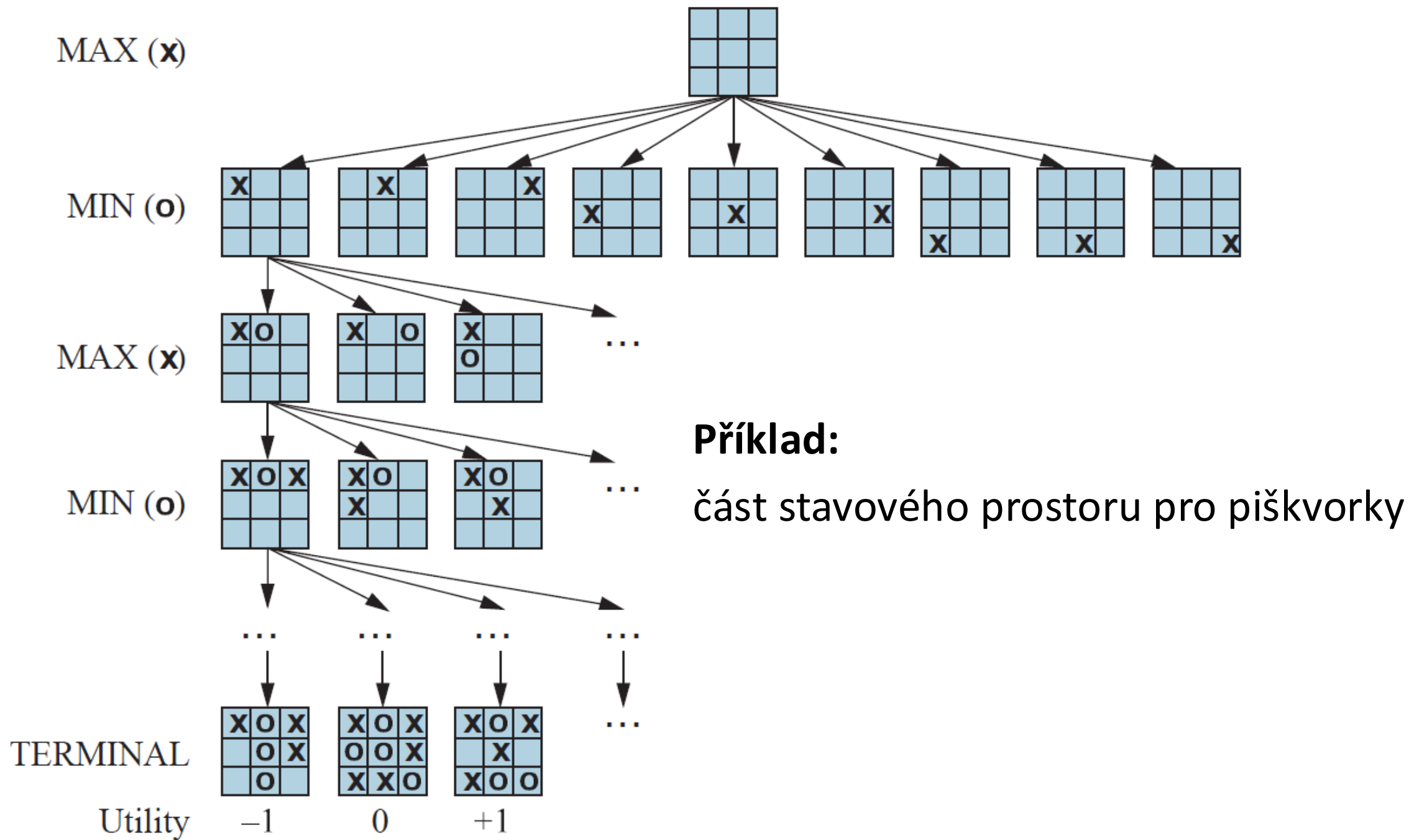
Tato přednáška se zabývá soutěživým prostředím, kde mají dva nebo více agentů protikladné cíle. To vede ke vzniku problémů soupeřivého prohledávání. Příslušné algoritmy je výhodné demonstrovat na hrách jako jsou šachy, Go a poker, neboť stav hry je snadno reprezentovatelný a agenti jsou obvykle omezeni na malý počet akcí, jejichž účinky jsou definovány přesnými pravidly.

Soupeřivé agenty explicitně modelujeme pomocí techniky nazvané soupeřivé prohledávání herního stromu (adversarial tree search).

Základní prohledávací algoritmus - **minimax** - je zobecněním AND-OR prohledávání z přednášky č. 4.

HRY S NULOVÝM SOUČTEM PRO DVA HRÁČE

- prostředí je deterministické, plně pozorovatelné (s úplnou informací), se dvěma agenty
 - agenti (hráči), **MAX** a **MIN**, se střídají v akcích (tahích), začíná MAX
 - nulový součet – co je dobré pro jednoho hráče, je špatné pro druhého
 - návrh strategie je pro hráče MAX
 - hra je definována těmito prvky:
 - počáteční stav, stav = pozice
 - informace, kdo je na tahu ve stavu s
 - množina akcí pro stav s , akce = tah
 - přechodový model $M(s,a)$ – výsledek akce a použité ve stavu s
 - test konce hry
 - funkce užitku $U(s,p)$ – výsledná numerická hodnota pro hráče p , když hra skončí ve stavu s
 - $U(s,p)$ je funkce užitku (nazývaná též účelová funkce nebo výplatní funkce), která definuje konečnou číselnou hodnotu pro hráče p , když hra skončí v koncovém stavu s . Např. v šachu je výsledkem výhra, prohra nebo remíza s hodnotami 1, 0 nebo 1/2. Některé hry mají širší rozsah možných výsledků - například výplaty v backgammonu se pohybují od 0 do 192.
-



OPTIMÁLNÍ ROZHODOVÁNÍ VE HRÁCH

Princip: MAX chce najít posloupnost akcí vedoucích k vítězství, stejně tak i MIN.

Algoritmus se navržen pro výhru hráče MAX a předpokládá se, že MIN hraje také optimálně.

MAXova strategie musí být podmíněným plánem, tj. zohledňující každý z možných tahů MINa.

Ve hrách, které mají binární výsledek (výhra nebo prohra), lze k vytvoření podmíněného plánu použít prohledávání AND-OR (viz přednáška 4). U takových her je definice vítězné strategie pro hru totožná s definicí řešení nedeterministického plánovacího problému: v obou případech musí být žádoucí výsledek zaručen bez ohledu na to, co udělá "druhá strana".

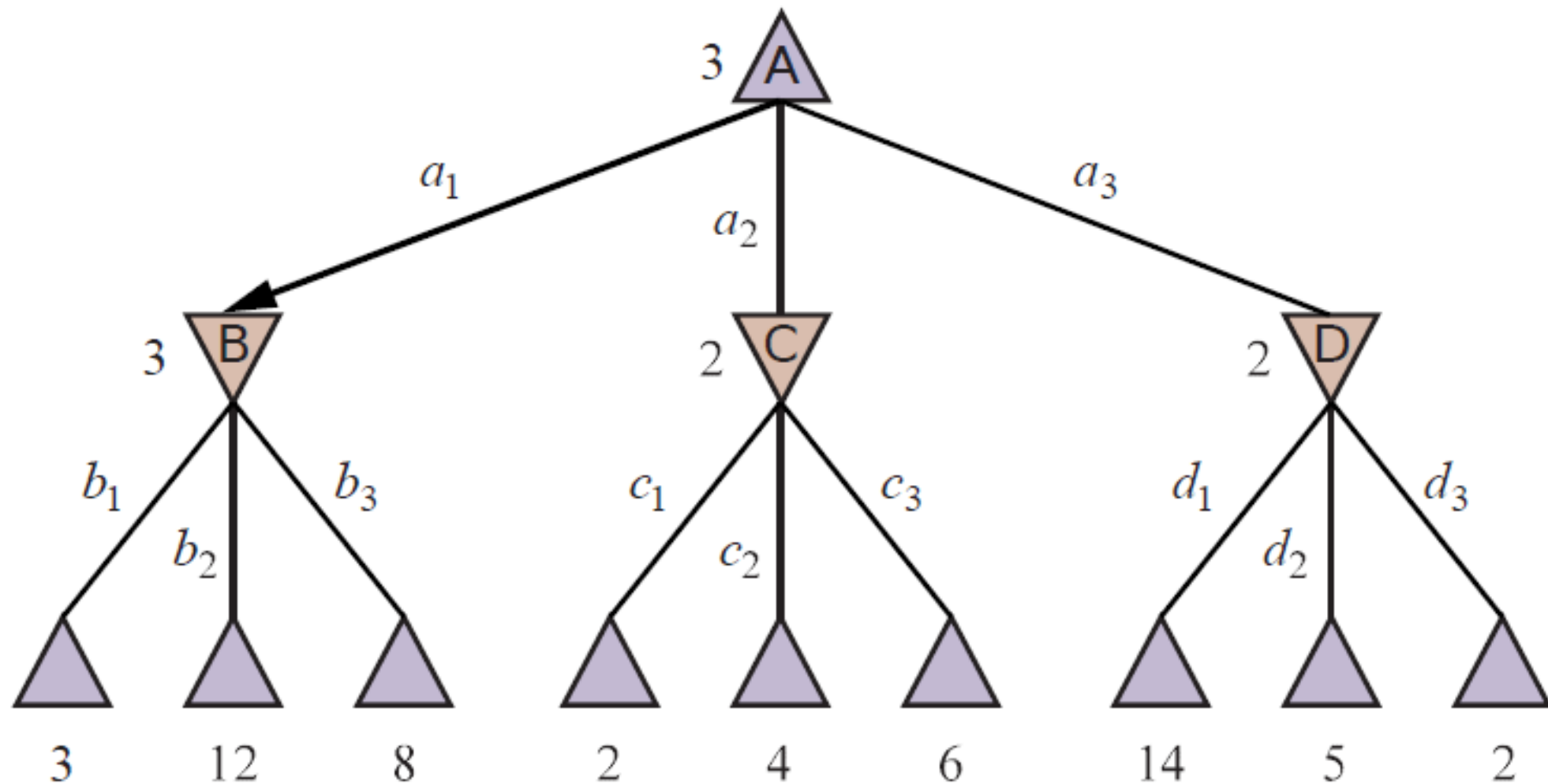
Pro hry s více výsledky se používá obecnější algoritmus nazývaný MINIMAX.

MiniMax provádí úplné prohledávání stromu hry do hloubky. Aktuální uzel expandujeme do dané hloubky d , uzly v hloubce d (listy) ohodnotíme a toto ohodnocení použijeme k odvození optimální akce v kořenu stromu (aktuálním uzlu). Toto je výpočetně zvládnutelné jen pro malé hloubky.

OPTIMÁLNÍ ROZHODOVÁNÍ VE HRÁCH

MAX

MIN



prohledávací algoritmus minimax

MINIMAX

PSEUDOKÓD

function MINIMAX-SEARCH(*game*, *state*) **returns** *an action*
 $\text{player} \leftarrow \text{game}.\text{TO-MOVE}(\text{state})$
 $\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state})$
 return *move*

function MAX-VALUE(*game*, *state*) **returns** *a (utility, move) pair*
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*
 $v \leftarrow -\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a))$
 if $v2 > v$ **then**
 $v, \text{move} \leftarrow v2, a$
 return *v, move*

function MIN-VALUE(*game*, *state*) **returns** *a (utility, move) pair*
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), *null*
 $v \leftarrow +\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a))$
 if $v2 < v$ **then**
 $v, \text{move} \leftarrow v2, a$
 return *v, move*

Pro příslušný herní strom lze optimální strategii určit tak, že pro každý stav ve stromu určíme hodnotu MINIMAX(s), což je užitek (pro hráče MAX) z pobytu v daném stavu za předpokladu, že oba hráči hrají optimálně od tohoto stavu až do konce hry.

Hodnota MINIMAXu koncového stavu je právě jeho užitek.

V nekoncovém stavu preferuje MAX tah do stavu s maximální hodnotou a MIN preferuje stav s minimální hodnotou (tj. minimální hodnota pro MAX, a tedy maximální hodnota pro MIN).

Algoritmus MINIMAX provádí kompletní prohledávání stromu hry do hloubky. Pokud je maximální hloubka stromu ***m*** a v každém bodě je ***b*** legálních tahů, pak je časová složitost minimaxového algoritmu $O(b^m)$. Prostorová složitost je $O(bm)$ pro algoritmus, který generuje všechny akce najednou, nebo $O(m)$ pro algoritmus, který generuje akce po jedné.

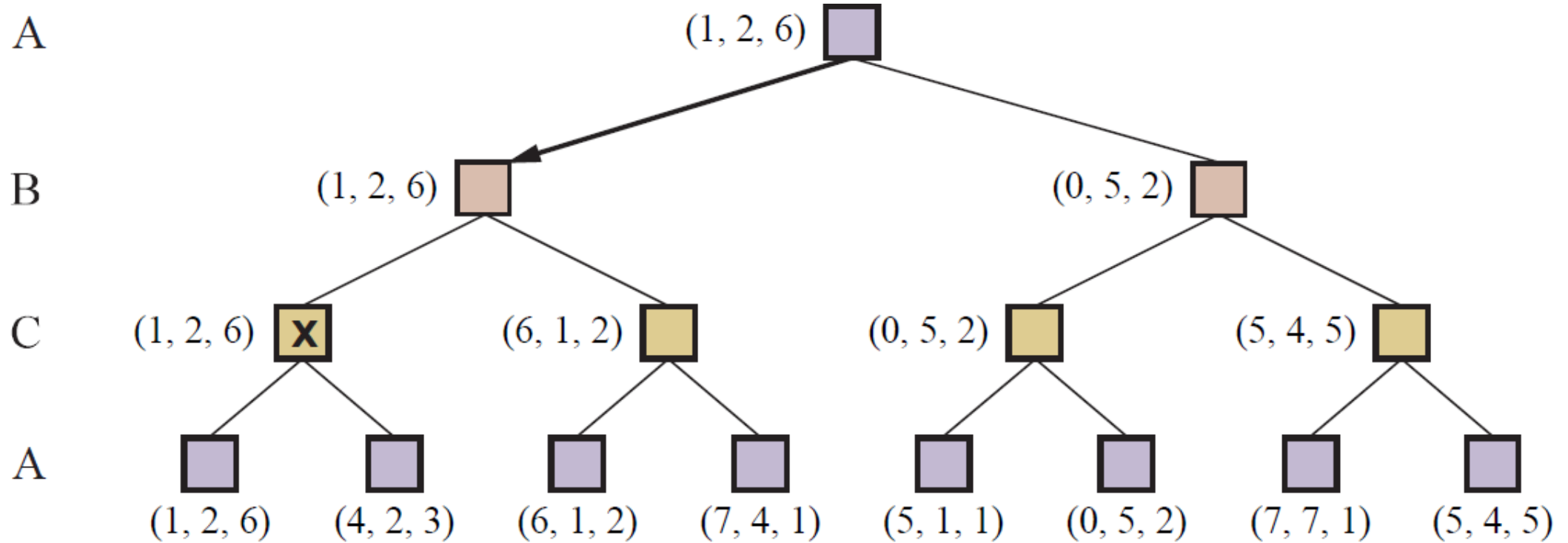
Kvůli této exponenciální složitosti je MINIMAX nepoužitelný pro složité hry; například šachy mají faktor větvení asi 35 a průměrná hra má hloubku asi 80 vrstev a prohledávání $35^{80} \sim 10^{123}$ stavů není proveditelné.

MINIMAX slouží jako základ pro matematickou analýzu her.

Aproximací této analýzy lze různými způsoby odvodit prakticky použitelné algoritmy.

OPTIMÁLNÍ ROZHODOVÁNÍ VE HRÁCH VÍCE HRÁČŮ

Hráč na tahu:



- funkce užitku jako vektor, hodnoty z pohledu každého hráče
 - možnost dočasných aliancí slabších hráčů
-

ALFA-BETA OŘEZÁVÁNÍ

- optimalizuje základní variantu algoritmu MiniMax
- ořezávání je ignorování těch větví pohledávacího stromu, které neobsahují optimální řešení
- v každém expandovaném uzlu se uchovávají dvě hodnoty:
 - α – užitek, který hráč MIN zaručeně nesníží při perfektní hře hráče MAX (užitek je „alespoň“) ...
 - β – užitek, který hráč MAX zaručeně nezvýší při perfektní hře hráče MIN (užitek je „nejvýše“) ...
... kdekoli na cestě od aktuálního uzlu směrem ke kořeni (včetně těchto uzlů)
- zlepšení pomocí alfa-beta ořezávání záleží na pořadí expanze uzlů

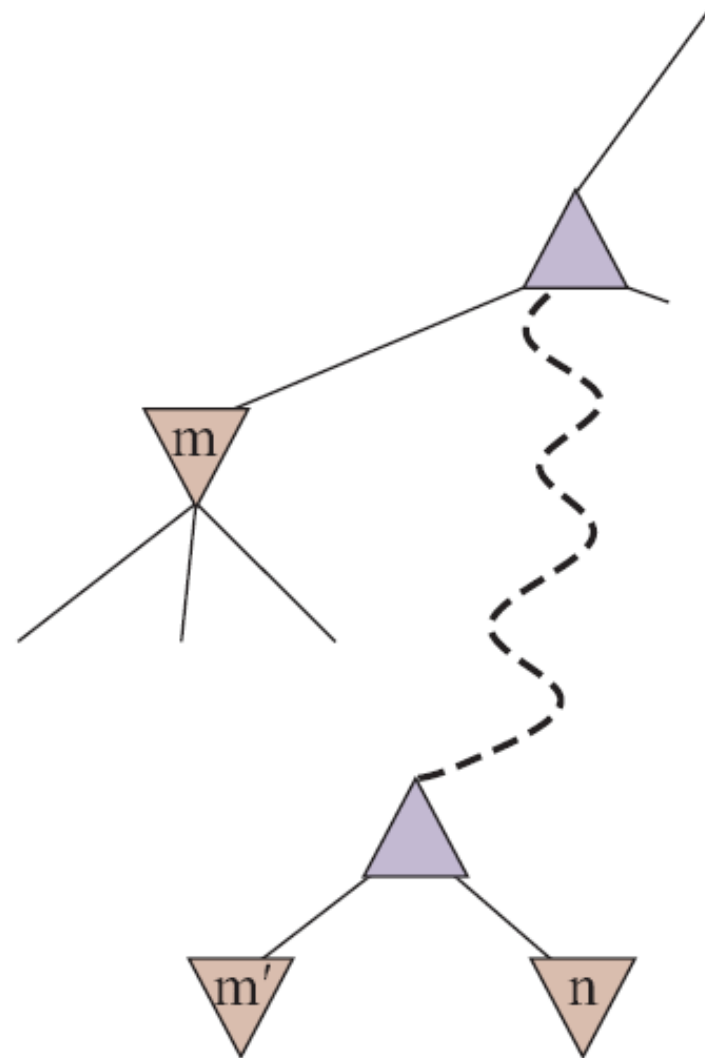
ALFA-BETA OŘEZÁVÁNÍ

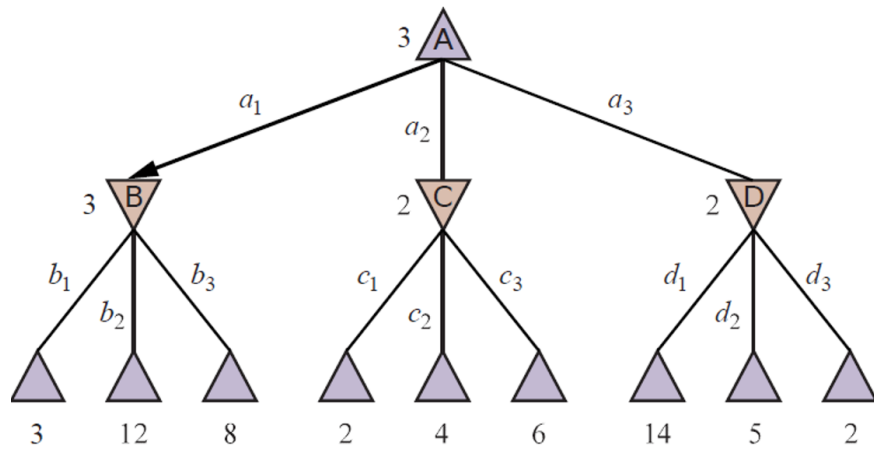
Alfa-Beta ořezávání lze aplikovat na stromy libovolné hloubky.

Oběcný princip: uvažujme, že někde ve stromu je uzel n , do kterého se hráč v daném tahu může přesunout.

Pokud má hráč lepší volbu buď na stejné úrovni (uzel m'), anebo na některé předchozí úrovni (uzel m), pak se nikdy nepřesune do n . Jakmile tedy hráč zjistí tyto lepší možnosti (prozkoumáním některých potomků uzlu n), může uzel n , a tedy celý příslušný podstrom, odřezat.

Závěr: Když víme, že nějaká větev je špatná (tj. horší než doposud nalezená nejlepší), nepotřebujeme už vědět, jak moc je špatná.

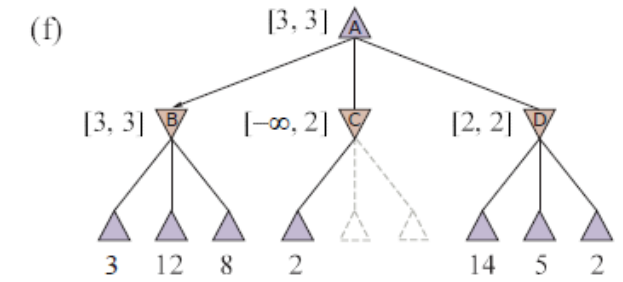
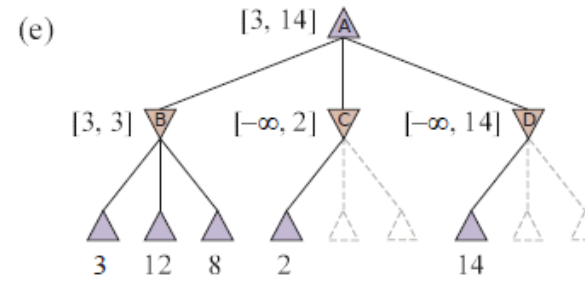
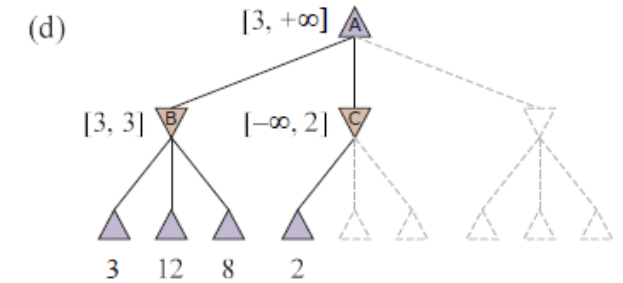
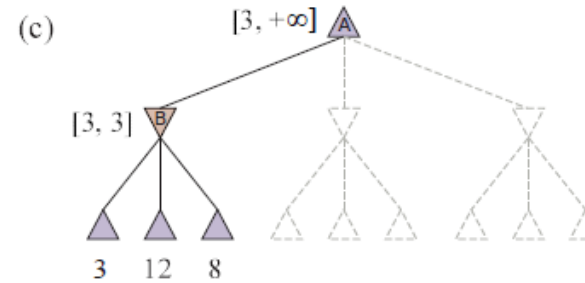
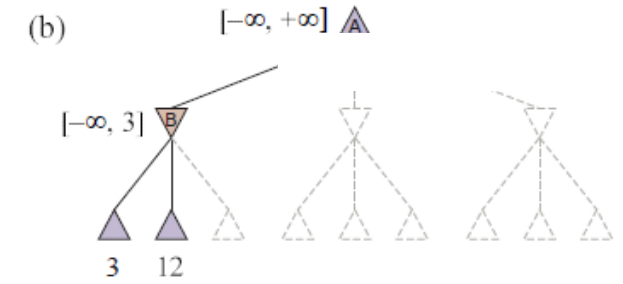
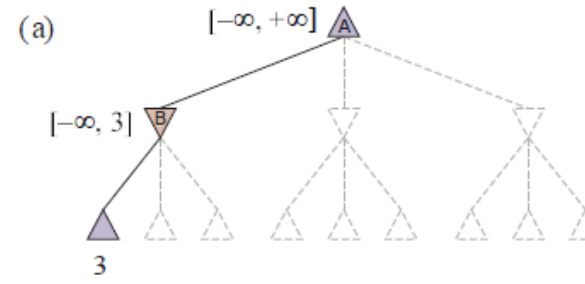




výpočet optimálního rozhodnutí z pohledu MAX

- (a)-(b) – po tahu MIN bude cena uzlu B nejvýše 3
- (c) - cena uzlu B je právě 3, odhad ceny uzlu A je alespoň 3
- (d) – C má hodnotu nejvýše 2; MAX ví, že B má cenu 3, a tedy uzel C nevybere => oříznutí
- (e) – D má cenu nejvýše 14 > 3, a tedy se zkoumají další následníci - dojde se k výsledku, že D má cenu nejvýše 2, a tedy proběhne návrat k uzlu B,

který má nejvyšší MinMax hodnotu



ALFA-BETA PROŘEZÁVÁNÍ

```
function ALPHA-BETA-SEARCH(game, state) returns an action  
  player  $\leftarrow$  game.TO-MOVE(state)  
  value, move  $\leftarrow$  MAX-VALUE(game, state,  $-\infty$ ,  $+\infty$ )  
  return move
```

```
function MAX-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow -\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MIN-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )  
    if v2 > v then  
      v, move  $\leftarrow$  v2, a  
       $\alpha \leftarrow$  MAX( $\alpha$ , v)  
    if v  $\geq \beta$  then return v, move  
  return v, move
```

```
function MIN-VALUE(game, state,  $\alpha$ ,  $\beta$ ) returns a (utility, move) pair  
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null  
  v  $\leftarrow +\infty$   
  for each a in game.ACTIONS(state) do  
    v2, a2  $\leftarrow$  MAX-VALUE(game, game.RESULT(state, a),  $\alpha$ ,  $\beta$ )  
    if v2 < v then  
      v, move  $\leftarrow$  v2, a  
       $\beta \leftarrow$  MIN( $\beta$ , v)  
    if v  $\leq \alpha$  then return v, move  
  return v, move
```

HEURISTICKÉ ALFA-BETA PROHLEDÁVÁNÍ - VARIANTA

Abychom dokázali využít omezený výpočetní čas, můžeme hledání přerušit dříve a na stavy aplikovat heuristickou vyhodnocovací funkci, kdy s nekoncovými uzly zacházíme, jako by byly koncové. Jinými slovy, funkci UTILITY nahradíme funkcí EVAL, která odhaduje užitečnost stavu. Nahrazujeme také test koncového stavu testem „mezního“ stavu, který musí pro koncové stavy vrátit hodnotu true, ale jinak se může libovolně rozhodnout, kdy hledání ukončí, a to na základě hloubky hledání a libovolné vlastnosti stavu, kterou se rozhodne zohlednit. To nám dává vzorec H-MINIMAX(s, d) pro heuristickou minimaxovou hodnotu stavu s při hloubce hledání d :

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if IS-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

HEURISTICKÉ ALFA-BETA PROHLEDÁVÁNÍ

- Heuristická vyhodnocovací funkce $EVAL(s, p)$ vrací odhad očekávané užitečnosti stavu s pro hráče p , stejně jako heuristické funkce z kapitoly 3 vracejí odhad vzdálenosti k cíli. Pro cílové stavy musí platit, že $EVAL(s, p) = UTILITY(s, p)$ a pro necílové stavy musí být ohodnocení někde mezi prohrou a výhrou.
- Co kromě těchto požadavků musí splňovat dobrá hodnotící funkce?
 - výpočet nesmí trvat příliš dlouho – cílem je zrychlit hledání
 - vyhodnocovací funkce by měla být silně korelovaná se skutečnými šancemi na výhru

HEURISTICKÉ ALFA-BETA PROHLEDÁVÁNÍ

- většina hodnotících funkcí pracuje s množinou příznaků stavů („features“) - např. v šachách počty různých figurek, "dobrá pěšcová struktura" a "bezpečnost krále" - příznakům je přidělena numerická hodnota; hodnoty všech vlastností se pak váženě sečtou a získá se hodnocení pozice --> vážená lineární funkce:

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- pokud nejsou příspěvky od jednotlivých příznaků nezávislé, je vhodnější použít nelineární funkci (např. 2 střelci v šachách jsou více než 2x cennější než jeden střelec)
 - pro cílové stavy je $\text{EVAL}(s, p)$ rovna funkci užitku, $\text{UTILITY}(s, p)$,
pro necílové stavy má hodnotu mezi prohrou a výhrou:
 $\text{UTILITY}(\text{prohra}, p) \leq \text{EVAL}(s, p) \leq \text{UTILITY}(\text{výhra}, p)$
-

HEURISTICKÉ ALFA-BETA PROHLEDÁVÁNÍ

- příznaky a váhy nejsou součástí pravidel, ale vychází ze zkušenosti hráčů
- nejsou-li zkušenosti, lze použít strojové učení
- zastavovací pravidlo – určuje podmínku pro ukončení prohledávání:
 - pevná hloubka
 - iterativní prohlubování - robustnější

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if IS-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

The next step is to modify ALPHA-BETA-SEARCH so that it will call the heuristic EVAL function when it is appropriate to cut off the search. We replace the two lines in Figure 5.7 that mention IS-TERMINAL with the following line:

if *game.IS-CUTOFF(state, depth)* **then return** *game.EVAL(state, player), null*

HEURISTICKÉ ALFA-BETA PROHLEDÁVÁNÍ

Dopředné ořezávání

- oproti alfa-beta ořezávání, které odstraňuje větve, které nemají vliv na výsledné ohodnocení, **dopředné ořezávání** odstraňuje tahy, které se sice zdají být špatné, ale ve skutečnosti mohou být dobré
 - pravděpodobnostní řez – používá statistiku z předchozích zkušeností a ořezává větve, které jsou **pravděpodobně** (nikoli na 100%) mimo okno (α, β)
 - technika redukce pozdějšího tahu – předpoklad, že tahy na konci fronty jsou spíše horší
 - úspora výpočetního času, ale riziko chyby
-

HEURISTICKÉ ALFA-BETA PROHLEDÁVÁNÍ

- mnohé herní programy před-počítávají tabulky nejlepších tahů pro zahájení a ukončení hry
- tabulky vychází ze zkušenosti hráčů
- vhodné pro začátek a konec hry
- **vyhledávání** = výběr vhodného tahu z této tabulky

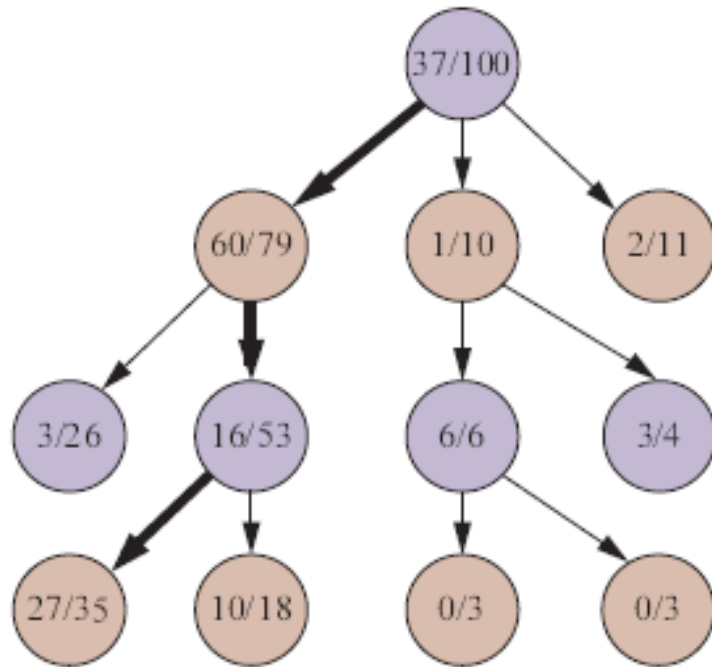
Prohledávání vs. Vyhledávání

- na začátku a konci hry se vhodné tahy vyhledávají v tabulce
 - uprostřed probíhá klasická prohledávání herního stromu
 - tento postup sníží rozměr herního stromu
-

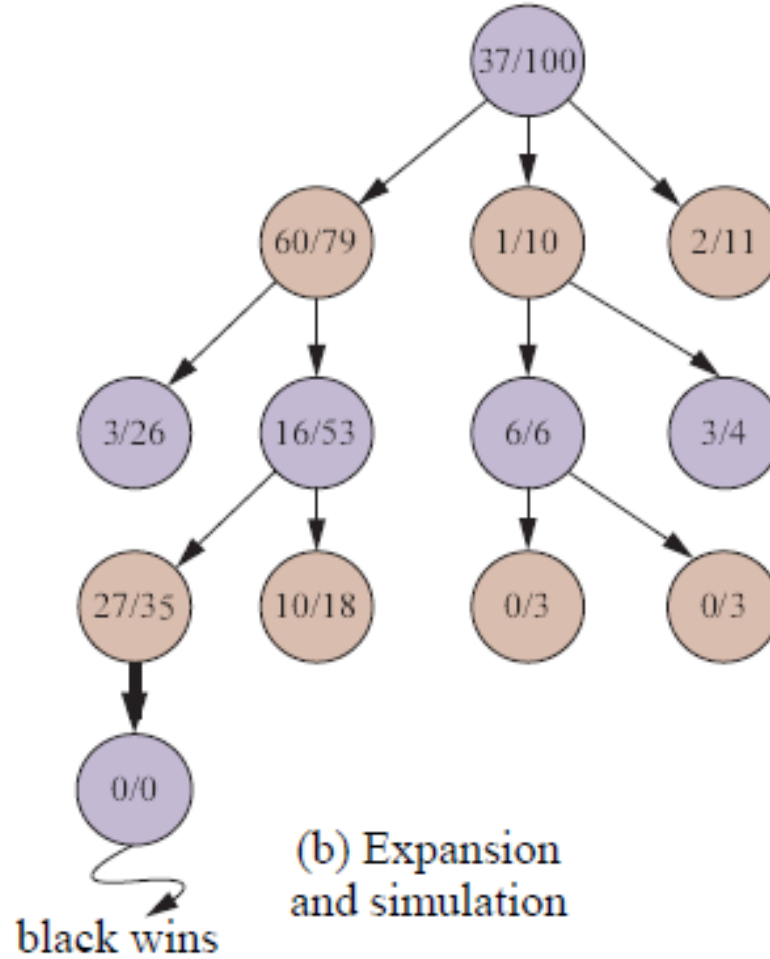
MONTE CARLO PROHLEDÁVÁNÍ (MCST)

- základní strategie MCTS nepoužívá heuristickou vyhodnocovací funkci - hodnota stavu se odhaduje jako průměrný užitek z řady simulací kompletních her, které začínají v tomto stavu
 - hra se odehraje podle pravidel až do konce a vyhodnotí se, kdo vyhrál - zvolené tahy nemusí být optimální; proces se opakuje vícekrát a dosažené výsledky se zprůměrují
 - simulace vybírá střídavě tahy pro oba hráče a pokračuje dokud není dosaženo konečné pozice
 - pro hry s výsledkem „výhra“ nebo „prohra“ se průměrný užitek rovná „procentu výhry“
 - herní politika = výběr vhodných tahů, poměr mezi průzkumem a využitím
 - volba stavu pro začátek simulace a počtu odehraných kol z tohoto stavu
 - MCTS udržuje strom hry a v každé iteraci provádí
 - výběr tahu na základě herní politiky
 - rozšíření – generování potomků vybraného uzlu
 - simulace – provedení hry z nově vygenerovaného potomka, tahy se neukládají do prohlídacího stromu
 - zpětná propagace – využití simulace k aktualizaci prohlídacího stromu až ke kořeni
-

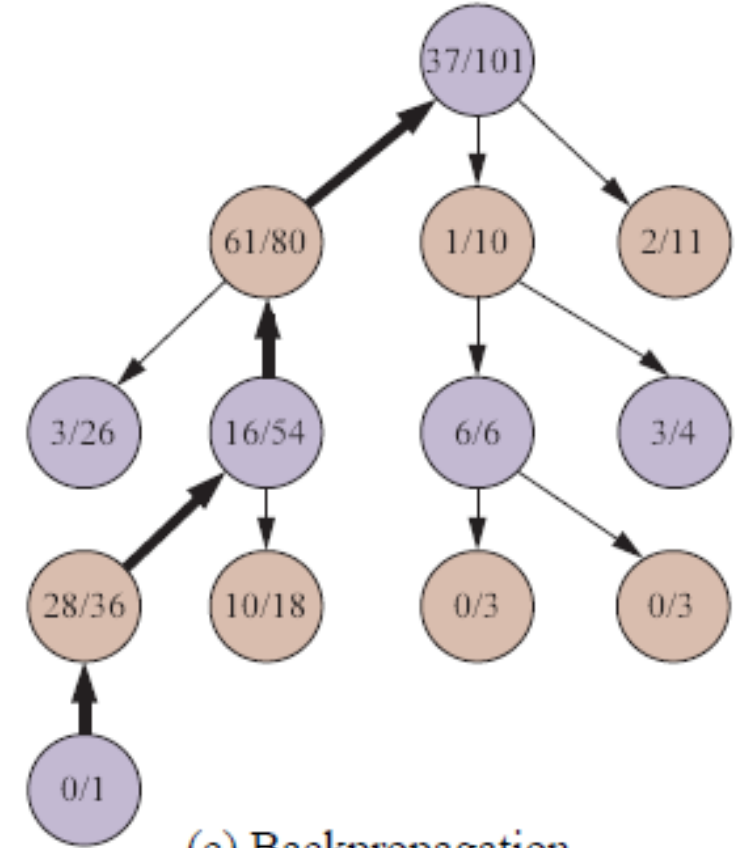
MONTE CARLO PROHLEDÁVÁNÍ - PŘÍKLAD JEDNÉ ITERACE



(a) Selection



(b) Expansion
and simulation



(c) Backpropagation

MONTE CARLO PROHLEDÁVÁNÍ - PSEUDOKÓD

function MONTE-CARLO-TREE-SEARCH(*state*) **returns** *an action*

tree $\leftarrow$ NODE(*state*)

while IS-TIME-REMAINING() **do**

leaf $\leftarrow$ SELECT(*tree*)

child $\leftarrow$ EXPAND(*leaf*)

result $\leftarrow$ SIMULATE(*child*)

 BACK-PROPAGATE(*result*, *child*)

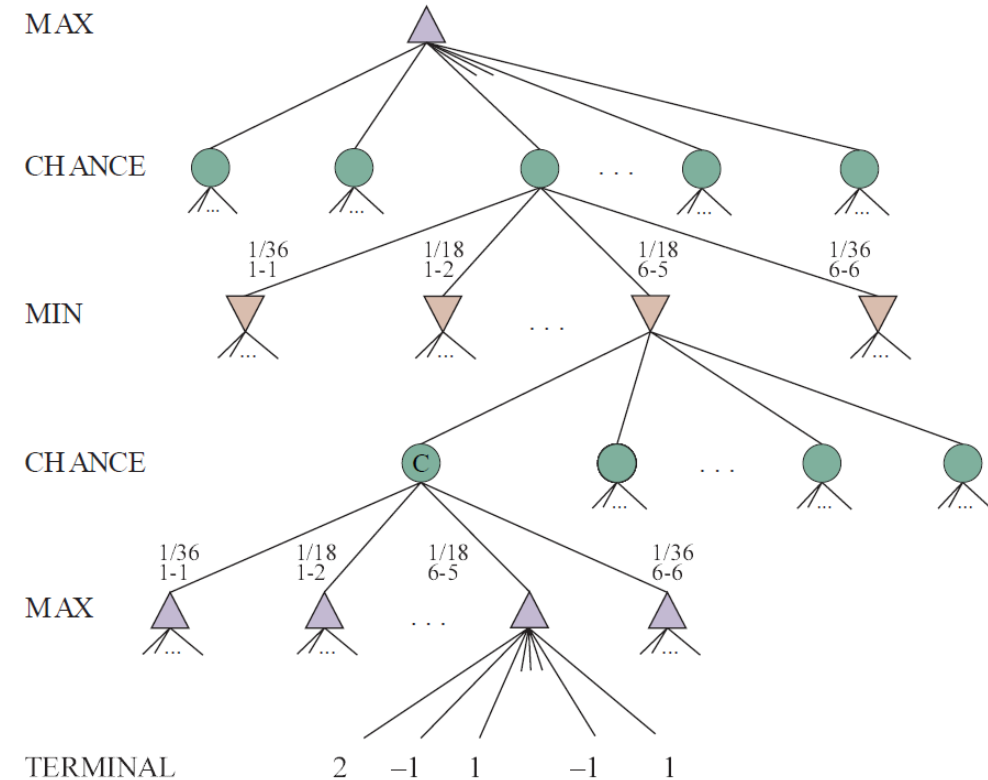
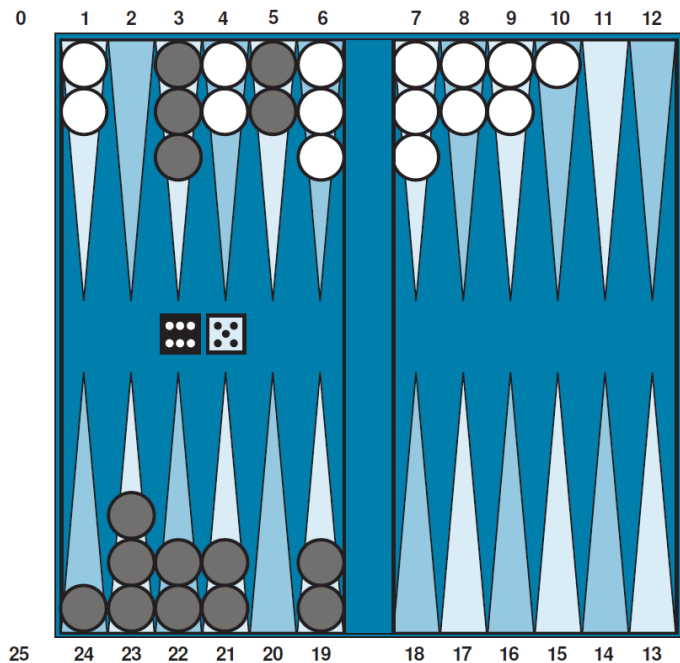
return the move in ACTIONS(*state*) whose node has highest number of playouts

LIMITY HERNÍCH PROHLEDÁVACÍCH ALGORITMŮ

- výpočet optimálních rozhodnutí ve složitých hrách je neřešitelný --> algoritmy musí být založeny na určitých předpokladech a aproximacích
 - Minimax s alfa-beta ořezáváním používá jako aproximaci heuristickou hodnotící funkci
 - Monte Carlo stromové prohledávání (MCTS) vypočítává přibližný průměr přes náhodný výběr her
 - výběr algoritmu závisí na vlastnostech každé hry - pokud je faktor větvení vysoký nebo je obtížné definovat hodnotící funkci, používá se MCTS
-

STOCHASTICKÉ HRY

- obsahují prvek náhody (hod kostkou)
- strom hry obsahuje min-max uzly + uzly náhody
→ algoritmus expect-minimax (rozšířená verze minimaxu), který vyhodnocuje náhodný uzel tak, že uvažuje vážený průměr funkce užitku všech jeho potomků, kdy váhy odpovídají pravděpodobnosti jednotlivých potomků
- počítá se očekávaná hodnota dané pozice



HRY V ČÁSTEČNĚ POZOROVATELNÉ

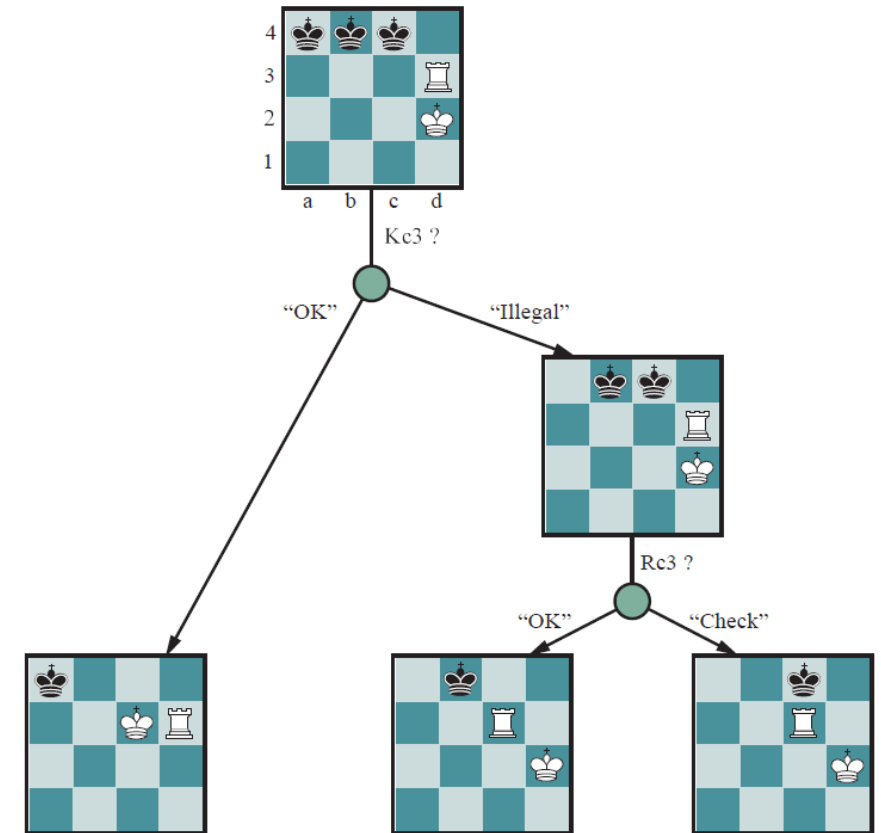
Kriegspiel

- varianta šachů, kdy hráč vidí pouze své vlastní figurky
- rozhodčí vidí vše a informuje oba hráče o legálnosti jejich tahů

Karetní hry

- na začátku prvek náhody (zamíchání karet)
- použití dopředného prořezávání

Optimální hra vyžaduje odhad očekávaného současného i budoucího stavu každého hráče. Jednoduchou aproximaci lze získat zprůměrováním hodnoty akce přes všechny možné konfigurace chybějících informací.



SHRNUTÍ

- Hra je definována počátečním stavem, povolenými akcemi (tahy) pro každý stav, výsledkem každé akce, testem konce hry, užítkovou funkcí, která se aplikuje na terminální stavy a říká, kdo vyhrál a jaké je konečné skóre.
- V diskrétních, plně pozorovatelných, deterministických hrách pro dva hráče s nulovým součtem se používá algoritmus minimax, který vybírá optimální tahy prohledáváním do hloubky, vytváří se strom hry.
- ořezávání alfa-beta vypočítá stejný optimální tah jako minimax, ale dosahuje mnohem větší efektivity tím, že odstraňuje podstromy, které jsou prokazatelně neoptimální.
- obvykle není možné (ani pomocí alfa-beta) uvažovat celý herní strom, takže je prohledávání v určitém bodě zastaveno a použije se heuristická hodnotící funkce, která odhaduje užítkovou funkci stavu.

Zdroj:

Russell, S. J., Norvig, P. *Artificial Intelligence: A
Modern Approach*. 4. vyd. Pearson, 2021
